Prompt to Production: AI Agent Mini-Competition

Student Brief & Problem Definition

Welcome to the *Prompt to Production* challenge. Over the next 48 hours, you will design, optimize, and deploy an autonomous AI agent capable of solving a complex, hybrid data science problem.

You are not just writing a script; you are building an agentic workflow that must reason through unstructured data, write/execute code, and survive the chaos of a production deployment on Railway.

Your opponent? A benchmark solution deployed by the instructor using the out-of-the-box autonomous **Hermes Agent**. Your goal is to build a custom **LangChain/LangGraph** architecture that is faster, more resilient, and more accurate.

1. The Problem: Luxury Real Estate Tiering

Your objective is to predict the price tier of New York Airbnb listings.

Traditional tabular Machine Learning models (like XGBoost or Random Forest) are great at analyzing numbers (e.g., minimum_nights, availability_365). However, in real estate, the true value of a property is often hidden within the messy, unstructured marketing descriptions written by hosts (e.g., *"penthouse view"*, *"needs minor repairs"*, *"premium Italian marble"*).

Your Mission: Build an agent that can intelligently read these unstructured descriptions, extract the hidden value indicators, combine them with the tabular data, and predict the correct property tier.

The target variable is price_tier, categorized as:

- 0: Budget
- 1: Standard
- 2: Premium
- 3: Ultra-Luxury

2. The Dataset

You have been provided with two files: train.csv and test.csv.

Data Schema

The dataset contains a rich set of features. Depending on the exact data dump, your CSV will include a variety of tabular and textual columns. The most important ones to utilize include:

- property_id: Unique identifier for the listing.
- **Tabular Metrics:** Various numeric and categorical columns detailing the property (e.g., room_type, neighbourhood, minimum_nights, number_of_reviews, availability_365).
- description: **The critical feature.** Unstructured text written by the host (may also appear under similar naming conventions like name or neighborhood_overview).
- price_tier: The target variable (0, 1, 2, or 3). **Note: This is only available in train.csv.**

The Hidden Validation Set

At the end of Day 2, the instructor will pass a secret validation.csv to your deployed Railway application.

Warning: This validation set is sabotaged. It contains "curveballs" such as missing tabular data (which your agent must infer from the text), altered column names, and multi-lingual descriptions. A rigid Python script will crash. An autonomous agent will adapt.

3. The Technology Stack & Constraints

To ensure a level playing field, you must adhere to the following constraints:

1. **Local Development Environment:** You must use the provided **Devcontainer**. This ensures everyone is working in the exact same Linux environment, eradicating "it works on my machine" issues.
2. **Local LLM Inference:** During Day 1 development, you **must** use local models via **Ollama** (e.g., Llama 3 8B or Phi-3). You cannot use paid GPT-4/Claude API keys to brute-force the logic. This forces you to write better tools and better prompts.
3. **Agent Framework:** You will use **LangChain** and **LangGraph** to build your cognitive architecture, tool bindings, and state management.
4. **Production Deployment:** Your final agent must be deployed as a web application/endpoint on **Railway**, utilizing the Railway CLI.

4. How to Approach the Solution (The Battle Plan)

Do not start writing code immediately. Follow this pipeline:

Phase 1: Planning & Product Requirements Document (PRD)

Before touching Python, write a PRD for your agent.

- What tools will your agent need? (e.g., a tool to parse CSVs, a tool to run scikit-learn models).
- How will the agent handle failures? If the dataset changes shape, what is the fallback loop in your LangGraph state?

Phase 2: Feature Engineering & Tool Creation

Your local LLM is not smart enough to do all the math in its head. You need to offload the heavy

lifting to deterministic Python code.

- Write a LangChain pipeline that uses the LLM *only* to read the description column and extract boolean flags (e.g., has_luxury_finishes = True, needs_renovation = False).
- Write a Python tool that takes those extracted flags, merges them with the standard tabular data, and trains a lightweight ML classifier.

Phase 3: Orchestration (LangGraph)

Wire your tools together. Define the conditional logic. Will your agent act linearly (read data $\rightarrow$ extract features $\rightarrow$ predict), or will it have a ReAct loop where it checks its own work and retries if a pandas operation fails?

Phase 4: Production (Railway)

Wrap your LangGraph agent in a simple API (e.g., FastAPI) and use the Railway CLI (railway up) to deploy it. Ensure your environment variables and API keys (for your production OpenRouter free-tier model) are securely configured.

5. Evaluation & Scoring

At the deadline, your Railway endpoints will be fed the hidden validation.csv. The winner will be determined by:

1. **Primary Metric - Macro F1-Score:** Because predicting the rare Ultra-Luxury tier is difficult, we use the Macro F1-score to measure overall accuracy across all 4 tiers.
2. **Secondary Metric - Resilience:** Did your agent successfully output a prediction file, or did it crash when it hit the instructor's injected data curveballs?
3. **Tie-Breaker - Efficiency:** If teams tie on accuracy, the winner is the team whose agent consumed the fewest LLM tokens to reach the answer.

Good luck. Plan carefully, prompt precisely, and code defensively.